Self Assignment Questions, Solved

Database Systems, COMSATS Lahore. Every assignment question from Lectures 4, 9, 10, 13, 14 and 15, each followed by an answer that is short, simple and easy to memorise.

How every answer is built

- **Short answer** (green box): the answer in one or two lines. If the examiner gives 2 minutes, write this.
- **Steps or reasons**: the same answer opened up so you understand why.
- **Memorise it** (gold box): one line or a mnemonic to remember it.

Questions in this file

No.	Lecture	Question (from the slide)
1	4	Prepare Level 0 and Level 1 diagram (bus garage DFD)
2	9	How to change the value of a primary key for an entity?
3	9	How to change the primary key for a table?
4	10	Can attributes other than the PK (non-key attributes) be used as a foreign key?
5	10	How to drop a tuple in the parent table when it is referred to in the child table?
6	10	What is the minimum FK eligibility criteria for a column?
7	13	Translation of key constraints to the relational model (1:1, 1:Many, Many:1, Many:Many)
8	14, 15	How to map enterprise hierarchies in an ERD?
9	14, 15	What is the major source of information, and how can it be captured?
10	14, 15	How many entities and relationships are mandatory for an ERD?
11	14, 15	Is an ERD a formal representation?
12	14, 15	How to standardize an ERD?
13	14	Assignment: design an ER diagram for the microscope reservation system

These are my worked answers from the lecture material. Your instructor may expect slightly different wording, so use them to understand, then write in your own words.

LECTURE 4

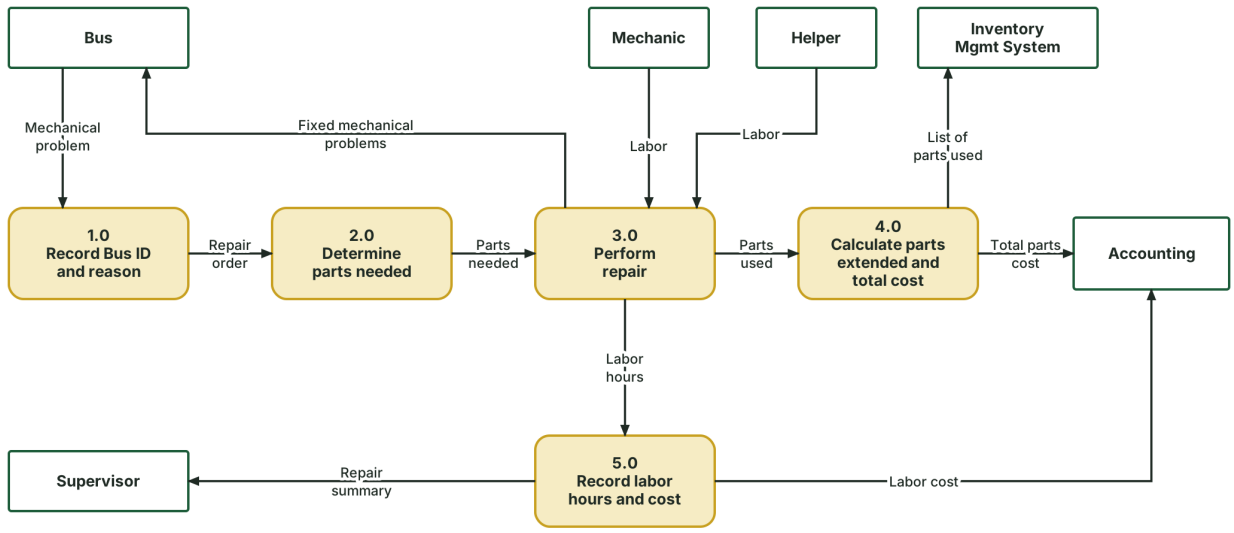
1 Prepare the Level 0 and Level 1 diagram for the bus garage repair system.

Short answer: Level 0 opens the single system bubble into the **five processes** from the slide. Level 1 opens **one** of those processes and now shows the **data stores**.

How to get there (4 steps)

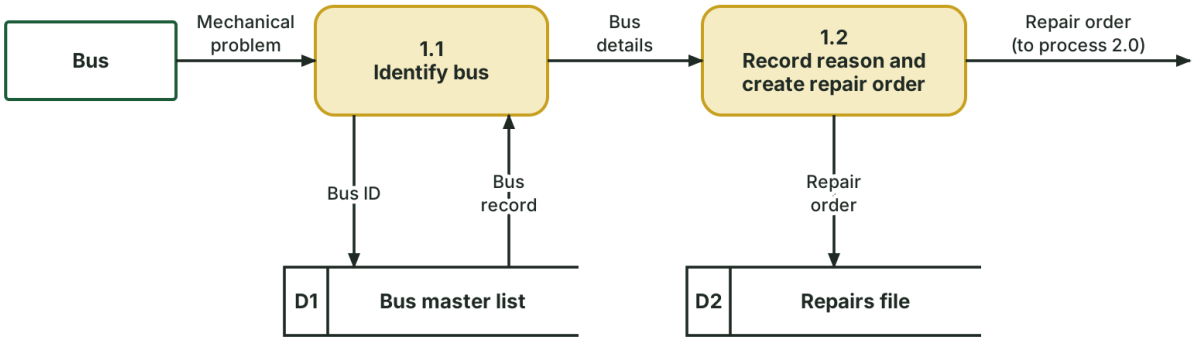
1. **Collect the parts** from the slide: 6 external entities, 5 processes, 4 data stores, 5 data flows (table below).
2. **Level 0:** draw the five processes in repair order, number them 1.0 to 5.0, put the entities around them, name every arrow. No stores yet.
3. **Level 1:** pick one process and split it into 1.1, 1.2 and so on. Now draw the stores it reads or writes.
4. **Check:** every process has an input and an output, arrows are named with data, and no arrow joins two entities or two stores directly.

Slide item	List from the slide
External entities	Bus, Mechanic, Helper, Supervisor, Inventory Management System, Accounting Department
Processes	1.0 Record Bus ID and reason, 2.0 Determine parts needed, 3.0 Perform repair, 4.0 Calculate parts extended and total cost, 5.0 Record labor hours and cost
Data stores	Personnel file, Repairs file, Bus master list, Parts list
Data flows	Repair order, Bus record, Parts record, Employee timecard, Invoices



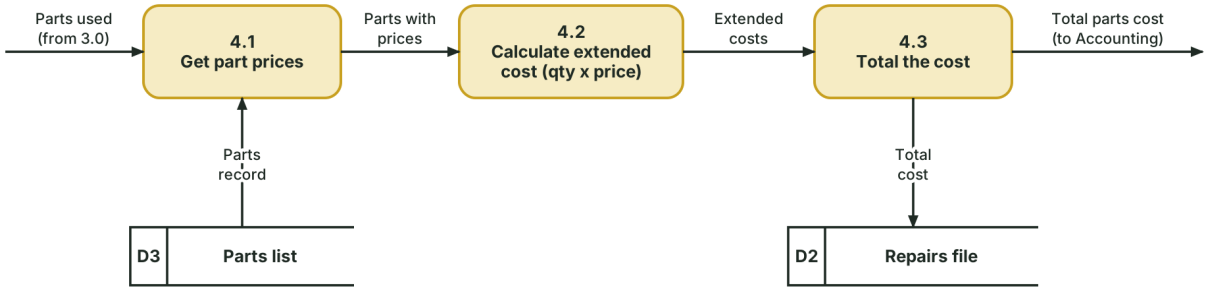
Level 0: the five processes of the slide. No data stores yet (they appear at level 1).

Level 0 DFD (answer). Read it left to right: problem comes in, repair is done, costs go to Accounting.



Level 1 of process 1.0. Stores D1 and D2 appear for the first time.

Level 1 of 1.0. Here Bus master list (D1) and Repairs file (D2) appear.



Level 1 of process 4.0. Process 5.0 is opened the same way.

Level 1 of 4.0. It reads Parts list (D3) and writes Repairs file (D2). Together with 5.0 it produces the invoice data for Accounting.

Process 5.0 opens the same way: **5.1** read the Employee timecard, **5.2** get the pay rate from the Personnel file, **5.3** calculate labor cost and write it to the Repairs file.

Memorise it: Level 0 = the main bubbles. Level 1 = open one bubble, stores appear. **Nouns sit** (entities, stores), **verbs move** (processes).

2 LECTURE 9, SELF ASSIGNMENT 1

How to change the value of the primary key for an entity?

Short answer: Run an **UPDATE** on that row. But every child row that points to it must change too, so use **ON UPDATE CASCADE** or update the children yourself.

Why and how

1. A primary key identifies one row, so the new value must be **unique and not NULL** (entity integrity).
2. Check which tables have a **foreign key** pointing at this key. They would be left pointing at nothing.
3. If the foreign key was created with **ON UPDATE CASCADE**, one UPDATE fixes the children automatically.

4. If the key is an **IDENTITY** (auto number) column in SQL Server, it cannot be updated at all. Insert a new row with the new value instead and move the children to it.

```
-- change one customer's key from 1001 to 2001
UPDATE Persons SET ID = 2001 WHERE ID = 1001;

-- let the children follow automatically
ALTER TABLE Orders
  ADD CONSTRAINT FK_Orders_Persons
  FOREIGN KEY (PersonID) REFERENCES Persons(ID)
  ON UPDATE CASCADE;
```

Memorise it: Change the row with UPDATE, make the kids follow with CASCADE. Best advice: a primary key should **never** change, that is why surrogate keys exist.

3 LECTURE 9, SELF ASSIGNMENT 2

How to change the primary key for a table?

Short answer: Drop the old primary key constraint, then **add** a new one. Unlink the child tables first and relink them at the end.

Steps, in this order

1. **Drop the foreign keys** in other tables that reference the old primary key (SQL Server will refuse otherwise).
2. **Drop the old primary key** constraint using its name.
3. **Prepare the new key:** the column must be **NOT NULL** and every value must be unique.
4. **Add the new primary key.**
5. **Recreate the foreign keys** so they point to the new key.

```
ALTER TABLE Orders  DROP CONSTRAINT FK_Orders_Persons;   -- 1
ALTER TABLE Persons DROP CONSTRAINT PK_Person;            -- 2
ALTER TABLE Persons ALTER COLUMN Email varchar(255) NOT NULL;  -- 3
ALTER TABLE Persons ADD CONSTRAINT PK_Person PRIMARY KEY (Email);  -- 4
-- 5: recreate the FK so it points at the new key
```

Memorise it: Unlink, remove, check, add, relink. Think of changing a lock: take the old lock out, make sure the new one fits, put it in, then give everyone new keys.

4 LECTURE 10, SELF ASSIGNMENT 1

Can attributes other than the primary key (non-key attributes) be used as a foreign key?

Short answer: Yes. A foreign key does not have to be a key in its own table, and it can point at any **candidate key** (a UNIQUE column) in the parent, not only the primary key.

The two sides, explained

- **Child side:** the FK column is usually an ordinary non-key column. In Orders, PersonID is not the key of Orders, it is just a link. It can also be *part* of a composite primary key (for example Enrolment(StudentID, CourseID)).

- **Parent side:** by the lecture definition, an FK matches the **candidate key** of another relation. So the parent column can be the primary key *or* any UNIQUE column. A column that is not unique cannot be referenced.

```
CREATE TABLE Person (
    PersonId int PRIMARY KEY,
    Email    varchar(100) NOT NULL UNIQUE );   -- candidate key

CREATE TABLE Orders (
    OrderID     int PRIMARY KEY,
    PersonEmail varchar(100),                  -- non-key column in Orders
    FOREIGN KEY (PersonEmail) REFERENCES Person(Email) );
```

Memorise it: An FK points at **any candidate key** (PK or UNIQUE). The FK column itself can be a plain, non-key column.

LECTURE 10, SELF ASSIGNMENT 2

5 How to drop a tuple (row) in the parent table when it is referred to in the child table (FK)?

Short answer: By default the DBMS **refuses** the delete (referential integrity). You must either delete the child rows first, or define the FK with an **ON DELETE** action.

Your options

Option	What happens to the child rows	When to use
Delete children first	You remove them yourself, then delete the parent	One-off clean up
ON DELETE CASCADE	Children are deleted automatically with the parent	Children make no sense alone (order lines of a deleted order)
ON DELETE SET NULL	Child FK becomes NULL (column must allow NULL)	Children may live on without a parent
ON DELETE SET DEFAULT	Child FK becomes its DEFAULT value	A "no owner" placeholder exists
NO ACTION (default)	Delete is rejected with an error	Protect data

```
-- manual way: children first, then parent
DELETE FROM Orders WHERE PersonID = 1;
DELETE FROM Person WHERE PersonId = 1;

-- automatic way: define the FK once
FOREIGN KEY (PersonID) REFERENCES Person(PersonID)
    ON DELETE CASCADE
```

Memorise it: Kids first, or cascade, or null them out. Otherwise: refused.

6 LECTURE 10, SELF ASSIGNMENT 3

What is the minimum FK eligibility criteria for a column?

Short answer: A column can be a foreign key if it has the **same type** as the parent key, the parent column is **unique**, the parent table **exists**, and every value is **valid** (found in the parent or NULL).

The four conditions, with a memory letter each

Letter	Condition	In plain words
Type	Same data type (and compatible size) as the parent key column	int links to int, not to text
Unique parent	The column it refers to is a primary key or UNIQUE (candidate key)	You can only point at something that identifies exactly one row
Parent first	The parent table and key already exist	Create the parent before the child
Valid values	Each FK value exists in the parent key, or is NULL	No orphans, no fake references

Memorise it: TUPV: Type, Unique parent, Parent first, Valid values.

7 LECTURE 13, SLIDE MARKED "?"

Translate key constraints to the relational model: 1-to-1, 1-to-Many, Many-to-1, Many-to-Many.

Short answer: Decide **where the foreign key goes**. One-to-many: on the **many** side. One-to-one: on either side with UNIQUE. Many-to-many: a **separate table** holding both keys.

Rules (example: Employees, Departments, Manages)

Type	What you do	Where the FK goes
Many-to-1 (key constraint: each department has at most one manager)	Fold the relationship into the table on the many side	Departments gets ssn and since
1-to-Many	Same rule, mirrored	The table on the many side gets the FK
1-to-1	Put the FK in either table and make it UNIQUE (or merge the two tables)	Either side, plus UNIQUE
Many-to-Many	Create a new table for the relationship	The new table holds both keys as FKs, together as the PK

```
-- Many-to-1: Manages folded into Departments
CREATE TABLE Dept_Mgr (
    did INTEGER, dname CHAR(20), budget REAL,
    ssn CHAR(11), since DATE,
    PRIMARY KEY (did),
    FOREIGN KEY (ssn) REFERENCES Employees );

-- Many-to-Many: its own table
CREATE TABLE Manages (
    ssn CHAR(11), did INTEGER, since DATE,
    PRIMARY KEY (ssn, did),
    FOREIGN KEY (ssn) REFERENCES Employees,
    FOREIGN KEY (did) REFERENCES Departments );

-- 1-to-1: FK plus UNIQUE
ALTER TABLE Employees
    ADD passport_no INTEGER UNIQUE REFERENCES Passports(passport_no);
```

Memorise it: FK on the MANY side. One-to-one: either side plus UNIQUE. Many-to-many: own table.

8 LECTURES 14 AND 15

How to map enterprise hierarchies in an ERD?

Short answer: There are three kinds of hierarchy, and each has its own ER tool: a **chain of 1:M relationships** for levels, a **self relationship** for boss-of-boss, and **supertype and subtype** (inheritance) for kinds.

Kind of hierarchy	Example	How to draw it
Levels (containment)	Company > Department > Team > Employee	Each level is an entity, joined to the next by a **1:M** relationship
Reporting (boss over boss)	Manager > Employee > Trainee, all employees	One entity with a **self (recursive) relationship** and roles "manager" and "worker"
Types (kinds of the same thing)	Employee is a Manager or an Engineer	A **supertype with subtypes** (EERD, next lecture). Simple version: one entity with a type attribute

Memorise it: Chain for levels, loop for bosses, split for kinds.

LECTURES 14 AND 15

9 What is the major source of information for an ERD, and how can it be captured?

Short answer: The major source is the **users' requirements**: what the people who use the business say, do and write down. It is captured by **interviews, observation and reading documents**.

Ways to capture it

- **Ask:** interviews and questionnaires with managers and users.
- **Watch:** observe how the work is really done today.
- **Read:** existing forms, reports, old files and old systems (the invoice in Lecture 2 is a good example).
- **Write it down:** a requirements document or use cases, then underline the nouns (entities and attributes) and verbs (relationships).

Memorise it: Ask, Watch, Read, Write. The ERD is only as good as the requirements you collected.

LECTURES 14 AND 15

10 How many entities and relationships are mandatory for an ERD?

Short answer: Strictly: **at least 1 entity** and **0 relationships**. For a meaningful database: **at least 2 entities and 1 relationship**.

- An entity (with a key) is the only thing you must have. A relationship is not compulsory, a single entity set is already a valid diagram.
- A relationship needs at least one participating entity set. A **self relationship** uses just one entity set.
- A useful ERD links things together, so it normally has two or more entities and a relationship between them.
- Every entity should also have a **key** and every relationship needs its **cardinality**, those are the real mandatory details.

Memorise it: Minimum 1 entity. Sensible minimum 2 entities and 1 relationship.

11 LECTURES 14 AND 15

Is an ERD a formal representation?

Short answer: Semi-formal. It has fixed symbols and clear meaning and converts systematically into tables, but it has no single official standard or mathematical definition.

Why it looks formal	Why it is not fully formal
Fixed symbols: rectangle, diamond, ellipse, double line	Several notations exist (Chen, crow's foot, UML), books disagree
Clear rules for keys, cardinality, participation	The same story can be drawn in more than one valid way
Maps step by step to relational tables	No mathematical proof of correctness, unlike relational algebra

Memorise it: Strict symbols, loose rules = semi-formal.

12 LECTURES 14 AND 15

How to standardize an ERD?

Short answer: Agree on **one notation**, one set of **naming rules**, label everything the same way, keep a **data dictionary** and **review** every diagram against a checklist.

Standardization checklist

1. **One notation** for the whole team (Chen or crow's foot), and a legend on the diagram.
2. **Naming rules:** entities are singular nouns, relationships are verbs, attributes use consistent names.
3. **Same details on every diagram:** key underlined, cardinality (1, M, N) and participation (single or double line) on each relationship.
4. **Data dictionary:** one place describing every entity, attribute and domain.
5. **Review and tools:** peer review with a checklist, or a CASE tool that enforces the rules.

Memorise it: One notation, one naming rule, full labels, one dictionary, one review.

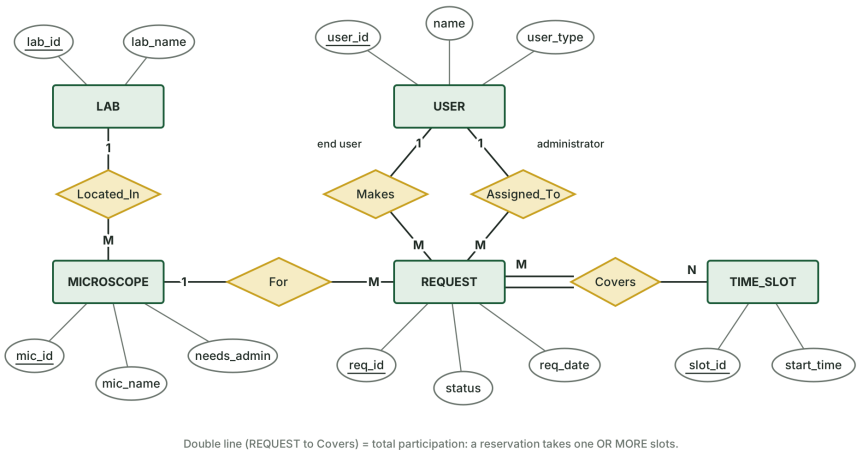
LECTURE 14, ASSIGNMENT

13 Design an ER diagram for an on-line reservation system for microscopes in a material science lab.

Short answer: Five entities (**User, Microscope, Lab, Request, Time_Slot**) and five relationships. A request is made by an end user, assigned to an administrator, is for one microscope, and covers one or more time slots.

Step 1: nouns to entities, verbs to relationships (the recipe from Lecture 14)

Sentence in the question	What it gives us
Two types of users: administrators and end users	Entity **USER** with attribute user_type (admin or end user). Two roles in two relationships.
Each microscope is located in a specific lab	Entities **MICROSCOPE** and **LAB**, relationship **Located_In** (many microscopes, one lab)
Each request is assigned to an administrator who can authorize or deny it	Entity **REQUEST** with attribute status (pending, authorized, denied). Relationship **Assigned_To** (one admin, many requests)
Users make requests	Relationship **Makes** (one end user, many requests)
Using some microscopes requires an administrator	Attribute needs_admin (yes or no) on MICROSCOPE
Time is divided into 1 hour slots. A reservation takes one or more	Entity **TIME_SLOT**, relationship **Covers** (many to many, total participation for REQUEST)



ER diagram (Chen style). Key attributes are underlined, M and N mean many, the double line means every request must have at least one time slot.

Step 2: assumptions to write next to your diagram

- A request is for **one** microscope (several would make it many-to-many).
- Covers is many-to-many: a request has several slots, a slot appears in many requests (for different microscopes). A rule outside the diagram stops double booking of one microscope.
- If needs_admin is yes, the assigned administrator must be present.
- One USER entity with user_type keeps it simple. With inheritance (next lecture) you could use Administrator and End_User subtypes.

Memorise it: Nouns = boxes, verbs = diamonds, "one or more" = double line and M:N. Always write your assumptions next to the diagram.